

2. BuildTree: Basic Decision Tree vs. Random Forest

The central algorithmic difference is a *single modification* inside the recursive BUILDTREE procedure. Everything else — splitting criterion, stopping conditions, leaf predictions — is identical.

2.1 Side-by-Side Pseudocode

The two pseudocodes below are placed side-by-side to make the difference immediately visible. The **orange line** is the only change.

Basic BuildTree(I, k)

```
1: if  $|I| \leq k$  or all labels identical
   then
2:   return Leaf  $\hat{y}$  = majority class
3: end if
4:  $\text{Gini}^* \leftarrow \infty$ 
5: for each feature  $j$ , threshold  $s$  do
6:    $I^\pm \leftarrow$  partition of  $I$  on  $(j, s)$ 
7:    $c(j, s) \leftarrow$  wtd.  $\text{Gini}(I^-, I^+)$ 
8:   if  $c(j, s) < \text{Gini}^*$  then
9:      $(j^*, s^*) \leftarrow (j, s)$ 
10:  end if
11: end for
12: Partition  $I$  on  $(j^*, s^*)$ 
13: return Node( $j^*, s^*, \text{BT}(I^-), \text{BT}(I^+)$ )
```

RF BuildTree(I, k, m)

```
1: if  $|I| \leq k$  or all labels identical
   then
2:   return Leaf  $\hat{y}$  = majority class
3: end if
4:  $\text{Gini}^* \leftarrow \infty$ 
5: sample  $\mathcal{F} \subseteq [d]$ ,  $|\mathcal{F}| = m$ , w/o
replacement
6: for each  $j \in \mathcal{F}$ , threshold  $s$  do
7:    $I^\pm \leftarrow$  partition of  $I$  on  $(j, s)$ 
8:    $c(j, s) \leftarrow$  wtd.  $\text{Gini}(I^-, I^+)$ 
9:   if  $c(j, s) < \text{Gini}^*$  then
10:     $(j^*, s^*) \leftarrow (j, s)$ 
11:  end if
12: end for
13: Partition  $I$  on  $(j^*, s^*)$ 
14: return Node( $j^*, s^*, \text{BT}(I^-), \text{BT}(I^+)$ )
```

★ **The Single Change That Makes All the Difference.** Every node in the basic tree evaluates *all* d features. Every node in a Random Forest tree evaluates only m randomly sampled features, drawn freshly at that node. This one modification decorrelates the ensemble's trees.

Typical defaults: $m = \lfloor \sqrt{d} \rfloor$ (classification), $m = \lfloor d/3 \rfloor$ (regression).

3 The Algorithm

3.1 High-Level Idea

The k -means algorithm alternates between two steps until nothing changes:

1. **Assignment step:** Given the current centres $\mu_1, \dots, \mu_k$, assign each point to its nearest centre.
2. **Update step:** Given the current cluster assignments, recompute each centre as the mean of its assigned points.

Each step can only decrease (or maintain) the SSE, so the algorithm converges. It stops when the assignment step produces no changes.

3.2 Formal Algorithm

K-Means Clustering Algorithm

Input: A dataset of n points in $\mathbb{R}^d$, and the desired number of clusters k .

Initialise: Assign each point to a cluster at random. Compute the k initial centres, one per cluster, as the mean of the points in that cluster.

Repeat until no point changes cluster:

1. **Assignment step.** For each data point, compute its distance to every cluster centre and assign it to the cluster corresponding to the nearest centre. If any point's assignment changed, note that the algorithm has not yet converged.
2. **Update step.** For each cluster, recompute its centre as the mean of all points currently assigned to it:

$$\mu_\ell \leftarrow \frac{1}{m_\ell} \sum_{i \in \mathcal{C}_\ell} \mathbf{x}_i$$

where m_ℓ is the number of points now in cluster ℓ .

Return: the final cluster assignments and the k centres $\mu_1, \dots, \mu_k$.

Each step greedily reduces SSE without undoing the other's work.

- The **assignment step** keeps the centres fixed and moves each point to whichever centre is nearest — this can only reduce (or maintain) each point's squared distance contribution.
- The **update step** keeps the assignments fixed and asks: given the current members, what single point minimises the sum of squared distances to all of them? The answer is always the mean.

Because SSE decreases at every iteration and there are only finitely many ways to partition n points into k clusters, the algorithm must eventually stop.

3 The Algorithm

3.1 High-Level Idea

The k -means algorithm alternates between two steps until nothing changes:

1. **Assignment step:** Given the current centres $\mu_1, \dots, \mu_k$, assign each point to its nearest centre.
2. **Update step:** Given the current cluster assignments, recompute each centre as the mean of its assigned points.

Each step can only decrease (or maintain) the SSE, so the algorithm converges. It stops when the assignment step produces no changes.

3.2 Formal Algorithm

K-Means Clustering Algorithm

Input: Dataset $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$, number of clusters k .

Initialise: Choose an initial clustering C^0 (e.g. random assignment). Set each initial centre μ_ℓ^0 to be the centroid of the points in cluster ℓ under C^0 .

Set $t \leftarrow 0$, **converged** $\leftarrow$ **false**.

While not converged:

1. **converged** $\leftarrow$ **true**.

2. **Assignment step** — for each point $i \in \{1, \dots, n\}$:

- Set $C^{t+1}(i) \leftarrow C^t(i)$ (tentatively keep current assignment).
- For each cluster $\ell \neq C^{t+1}(i)$: if $\|\mathbf{x}_i - \mu_\ell^t\| < \|\mathbf{x}_i - \mu_{C^{t+1}(i)}^t\|$, then reassign: $C^{t+1}(i) \leftarrow \ell$ and set **converged** $\leftarrow$ **false**.

(Each point is assigned to its nearest current centre.)

3. **Update step** — for each cluster $\ell \in \{1, \dots, k\}$:

$$\mu_\ell^{t+1} = \frac{1}{|\{i : C^{t+1}(i) = \ell\}|} \sum_{i: C^{t+1}(i) = \ell} \mathbf{x}_i$$

(Set each centre to the mean of its newly assigned points.)

4. $t \leftarrow t + 1$.

Return: clustering C^t and centres μ^t .

3.3 Worked Example: Four Steps by Hand

K-Means on 9 Points in $\mathbb{R}^2$, $k = 2$

Dataset:

$$\mathbf{x}_1 = (1, 1), \quad \mathbf{x}_2 = (1, 3), \quad \mathbf{x}_3 = (2, 2), \quad \mathbf{x}_4 = (3, 1), \quad \mathbf{x}_5 = (3, 3)$$

$$\mathbf{x}_6 = (6, 2), \quad \mathbf{x}_7 = (7, 1), \quad \mathbf{x}_8 = (7, 3), \quad \mathbf{x}_9 = (8, 2)$$

Initialisation. $\mu_1 = (3, 3)$ and $\mu_2 = (3, 1)$

Initialisation — both centres near the middle

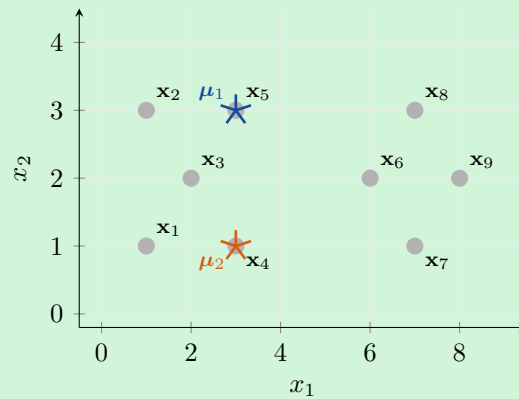


Figure 2: Initial state: all 9 points unassigned (grey). Both centroids start near the middle of the data.

Iteration 1 — Assignment step.

Point	Dist. to $\mu_1 = (3, 3)$	Dist. to $\mu_2 = (3, 1)$	Nearest	Assignment
$\mathbf{x}_1 = (1, 1)$	2.83	2.00	μ_2	Cluster 2
$\mathbf{x}_2 = (1, 3)$	2.00	2.83	μ_1	Cluster 1
$\mathbf{x}_3 = (2, 2)$	1.41	1.41	μ_1	Cluster 1 (tie $\rightarrow$ C1)
$\mathbf{x}_4 = (3, 1)$	2.00	0.00	μ_2	Cluster 2
$\mathbf{x}_5 = (3, 3)$	0.00	2.00	μ_1	Cluster 1
$\mathbf{x}_6 = (6, 2)$	3.16	3.16	μ_1	Cluster 1 (tie $\rightarrow$ C1)
$\mathbf{x}_7 = (7, 1)$	4.47	4.00	μ_2	Cluster 2
$\mathbf{x}_8 = (7, 3)$	4.00	4.47	μ_1	Cluster 1
$\mathbf{x}_9 = (8, 2)$	5.10	5.10	μ_1	Cluster 1 (tie $\rightarrow$ C1)

The result is a messy split: Cluster 1 = $\{\mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_8, \mathbf{x}_9\}$, Cluster 2 = $\{\mathbf{x}_1, \mathbf{x}_4, \mathbf{x}_7\}$ (bottom row only). Both initial centroids were so close together that the vertical position of each point almost entirely determined its assignment.

After Iter. 1 Assignment — messy horizontal split

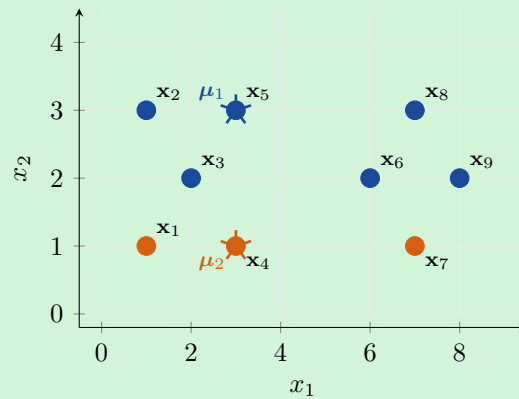


Figure 3: After Iteration 1 assignment. C1 (blue) holds the upper/middle-row points including the right-side points $\mathbf{x}_6, \mathbf{x}_8, \mathbf{x}_9$; C2 (orange) holds only the three bottom-row points.

Iteration 1 — Update step.

$$\mu_1 \leftarrow \frac{1}{6} [(1, 3) + (2, 2) + (3, 3) + (6, 2) + (7, 3) + (8, 2)] = (4.5, 2.5)$$

$$\mu_2 \leftarrow \frac{1}{3} [(1, 1) + (3, 1) + (7, 1)] = \left(\frac{11}{3}, 1\right) \approx (3.67, 1.00)$$

μ_1 is pulled far to the right by $\mathbf{x}_6, \mathbf{x}_8, \mathbf{x}_9$. μ_2 nudges right along the bottom.

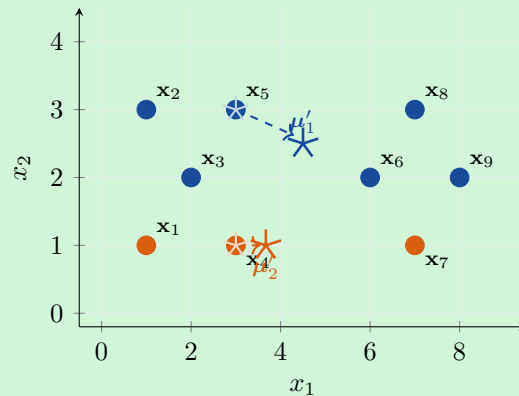
 After Iter. 1 Update — μ_1 jumps right to (4.5, 2.5)


Figure 4: After Iteration 1 update. μ_1 jumps to (4.5, 2.5), pulled by the right-side points in C1. μ_2 nudges to (3.67, 1).

Iteration 2 — Assignment step.

Point	Dist. to $\mu_1 = (4.5, 2.5)$	Dist. to $\mu_2 \approx (3.67, 1.0)$	Nearest	Assignment
$\mathbf{x}_1 = (1, 1)$	3.81	2.67	μ_2	Cluster 2
$\mathbf{x}_2 = (1, 3)$	3.54	3.33	μ_2	Cluster 2 $\leftarrow$ moved!
$\mathbf{x}_3 = (2, 2)$	2.55	1.94	μ_2	Cluster 2 $\leftarrow$ moved!
$\mathbf{x}_4 = (3, 1)$	2.12	0.67	μ_2	Cluster 2
$\mathbf{x}_5 = (3, 3)$	1.58	2.11	μ_1	Cluster 1
$\mathbf{x}_6 = (6, 2)$	1.58	2.54	μ_1	Cluster 1
$\mathbf{x}_7 = (7, 1)$	2.92	3.33	μ_1	Cluster 1 $\leftarrow$ moved!
$\mathbf{x}_8 = (7, 3)$	2.55	3.89	μ_1	Cluster 1
$\mathbf{x}_9 = (8, 2)$	3.54	4.45	μ_1	Cluster 1

Three points moved: $\mathbf{x}_2, \mathbf{x}_3$ left C1 for C2; $\mathbf{x}_7$ left C2 for C1. Now $C1 = \{\mathbf{x}_5, \mathbf{x}_6, \mathbf{x}_7, \mathbf{x}_8, \mathbf{x}_9\}$, $C2 = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4\}$.

After Iter. 2 Assignment — right side consolidating, $\mathbf{x}_5$ still in C1

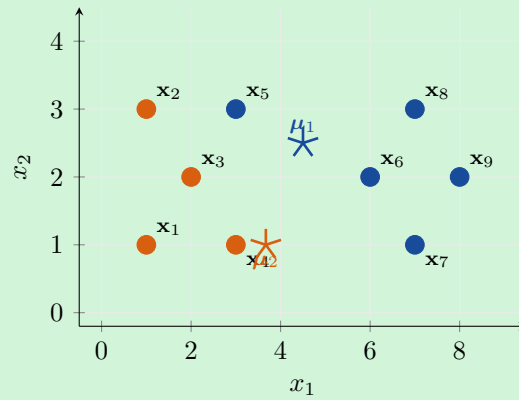


Figure 5: After Iteration 2 assignment. $\mathbf{x}_2, \mathbf{x}_3$ joined C2 (orange); $\mathbf{x}_7$ joined C1 (blue). Note $\mathbf{x}_5 = (3, 3)$ is still in C1 — it will switch in Iteration 3.

Iteration 2 — Update step.

$$\boldsymbol{\mu}_1 \leftarrow \frac{1}{5}[(3, 3) + (6, 2) + (7, 1) + (7, 3) + (8, 2)] = (6.2, 2.2)$$

$$\boldsymbol{\mu}_2 \leftarrow \frac{1}{4}[(1, 1) + (1, 3) + (2, 2) + (3, 1)] = (1.75, 1.75)$$

$\boldsymbol{\mu}_1$ is now in the right half; $\boldsymbol{\mu}_2$ has settled into the left cluster.

After Iter. 2 Update — centroids now on correct sides

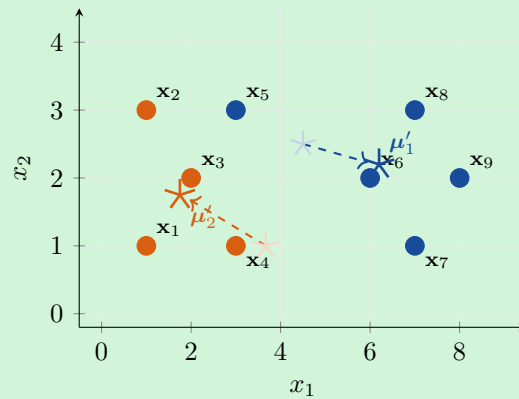


Figure 6: After Iteration 2 update. $\boldsymbol{\mu}_1$ moves right to $(6.2, 2.2)$; $\boldsymbol{\mu}_2$ moves left to $(1.75, 1.75)$. The centroids are now on the correct sides — but $\mathbf{x}_5 = (3, 3)$ will still switch.

Iteration 3 — Assignment step.

Point	Dist. to $\mu_1 = (6.2, 2.2)$	Dist. to $\mu_2 = (1.75, 1.75)$	Nearest	Assignment
$x_1 = (1, 1)$	5.34	1.06	μ_2	Cluster 2
$x_2 = (1, 3)$	5.26	1.46	μ_2	Cluster 2
$x_3 = (2, 2)$	4.20	0.35	μ_2	Cluster 2
$x_4 = (3, 1)$	3.42	1.46	μ_2	Cluster 2
$x_5 = (3, 3)$	3.30	1.77	μ_2	Cluster 2 $\leftarrow$ moved!
$x_6 = (6, 2)$	0.28	4.26	μ_1	Cluster 1
$x_7 = (7, 1)$	1.44	5.30	μ_1	Cluster 1
$x_8 = (7, 3)$	1.13	5.40	μ_1	Cluster 1
$x_9 = (8, 2)$	1.81	6.25	μ_1	Cluster 1

$x_5 = (3, 3)$ finally switches to C2. Now $C1 = \{x_6, x_7, x_8, x_9\}$ and $C2 = \{x_1, x_2, x_3, x_4, x_5\}$.

Iteration 3 — Update step.

$$\mu_1 \leftarrow \frac{1}{4}[(6, 2) + (7, 1) + (7, 3) + (8, 2)] = (7.0, 2.0)$$

$$\mu_2 \leftarrow \frac{1}{5}[(1, 1) + (1, 3) + (2, 2) + (3, 1) + (3, 3)] = (2.0, 2.0)$$

Both centroids are now at the exact geometric mean of their clusters.

After Iter. 3 Update — x_5 moved, centroids at true means

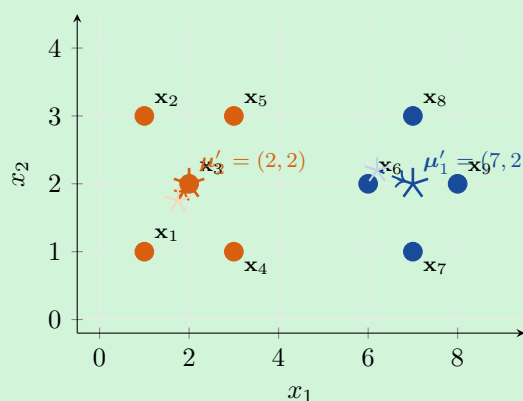


Figure 7: After Iteration 3 update. μ_1 settles at $(7, 2)$ and μ_2 at $(2, 2)$ — the exact means of the right and left clusters.

Iteration 4 — Assignment step (convergence check).

With $\mu_1 = (7, 2)$ and $\mu_2 = (2, 2)$, every point is closest to the same centroid as before. No point changes cluster. `converged = true`.

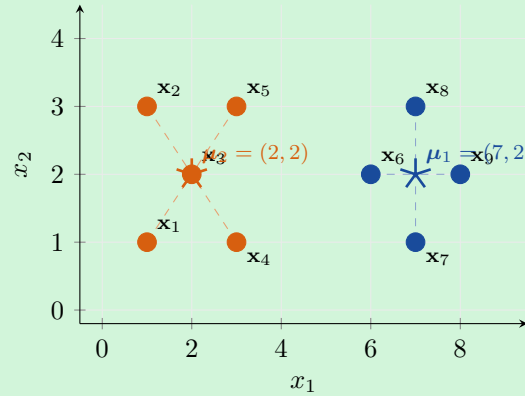
Final state — converged after 3 iterations

Figure 8: Final converged state. Dashed lines show each point's SSE contribution. The left/right separation is perfect.

Result. The algorithm converges after 3 real iterations. Final clusters:

- Cluster 1 (right): $\{x_6, x_7, x_8, x_9\}$, centre $\mu_1 = (7, 2)$.
- Cluster 2 (left): $\{x_1, x_2, x_3, x_4, x_5\}$, centre $\mu_2 = (2, 2)$.

Why did it take 3 iterations? Both centroids started in the middle, so Iteration 1 split the data horizontally (above vs below) rather than vertically (left vs right). After the first update, μ_1 drifted right and pulled most right-side points over, but $x_5 = (3, 3)$ stayed misassigned until Iteration 3 when μ_1 was far enough right to lose it.

Final SSE (exact integers):

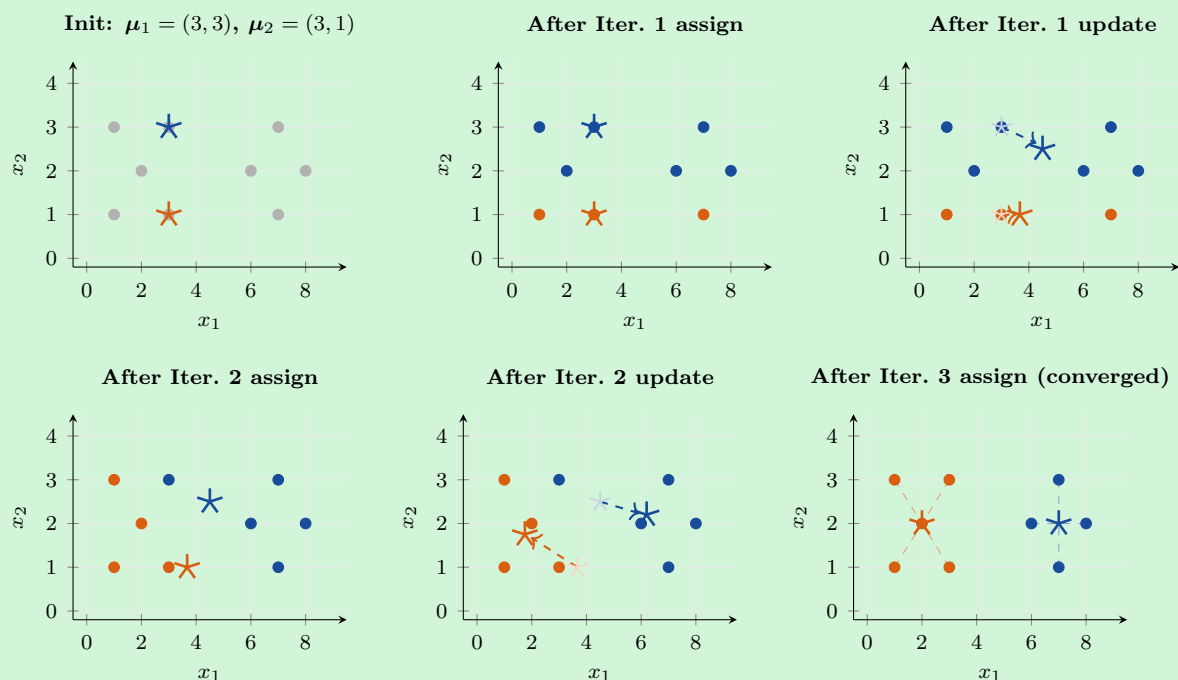
$$\text{C1: } \|x_6 - (7, 2)\|^2 + \|x_7 - (7, 2)\|^2 + \|x_8 - (7, 2)\|^2 + \|x_9 - (7, 2)\|^2 = 1 + 1 + 1 + 1 = 4$$

$$\begin{aligned} \text{C2: } & \|x_1 - (2, 2)\|^2 + \|x_2 - (2, 2)\|^2 + \|x_3 - (2, 2)\|^2 \\ & + \|x_4 - (2, 2)\|^2 + \|x_5 - (2, 2)\|^2 = 2 + 2 + 0 + 2 + 2 = 8 \end{aligned}$$

$$\text{SSE} = 4 + 8 = \mathbf{12}$$

Summary: how the clusters evolved across all steps.

Each panel below is the state *after* the assignment step of that iteration. Stars show the centroids *at the start* of that step; arrows (update panels) show where they moved.



Row 1 (left to right): initialisation $\rightarrow$ after Iter. 1 assignment $\rightarrow$ after Iter. 1 update (arrows show centroid shift).

Row 2: after Iter. 2 assignment $\rightarrow$ after Iter. 2 update $\rightarrow$ final state after Iter. 3 (SSE lines shown).